Software System Modelling & UML

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

27 February 2026

Today's Agenda

Agenda

Recap: Requirements & LEPSI

Last week, we moved from abstract planning to defining strict system boundaries and constraints.

- **Requirements Engineering:** Bridging the gap between what the customer wants and what the developers build.
- **Functional vs. Non-Functional:** What the system *does* vs. *how well* (or how securely) it does it.
- **LEPSI as Constraints:** We learned that Legal (GDPR), Ethical (Dual-Use), Professional, and Social (WCAG) issues act as strict Non-Functional Requirements.
- **Agile Implementation:** Translating requirements into User Stories with a strict **Definition of Done (DoD)**.

Today's Learning Objectives

By the end of this lecture, you should be able to:

1. **Explain** the purpose of software system modelling and the concept of abstraction.
2. **Identify** the standard UML (Unified Modeling Language) diagram types and their distinct perspectives.
3. **Map** Object-Oriented Programming (OOP) concepts directly to UML Class Diagrams.
4. **Select and apply** the appropriate models to visually represent a software system architecture.

Agenda

What is System Modelling?

Definition

System modelling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.

Why do we model?

- To help the analyst understand the functionality of the system.
- To communicate with customers (who may not understand code).
- To provide a blueprint for developers before coding begins.
- To serve as documentation for future maintenance and updates.

System Perspectives

A single model cannot capture everything. We use different models to represent different **perspectives**:

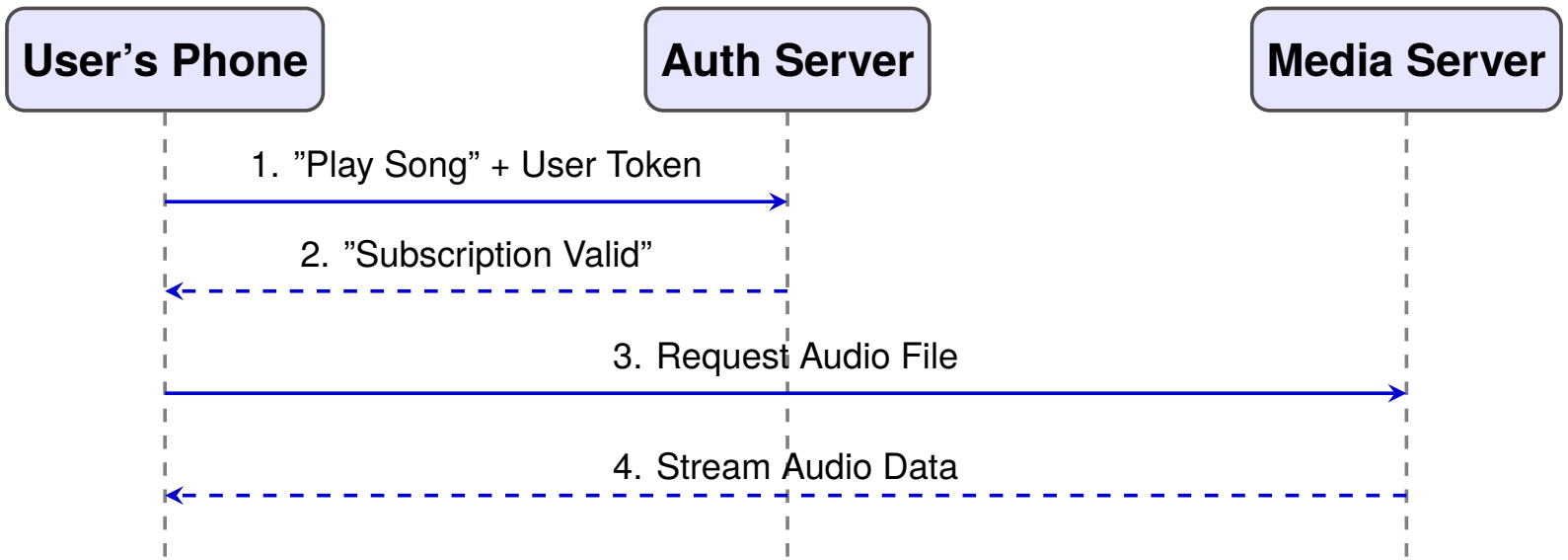
- **External perspective:** Models the context or environment of the system.
- **Interaction perspective:** Models the interactions between a system and its environment, or between its components.
- **Structural perspective:** Models the organization of a system or the structure of the data that is processed.
- **Behavioral perspective:** Models the dynamic behavior of the system and how it responds to events.

The Power of a Software Model

Abstraction in Action

A good model strips away the code to immediately communicate **how the system works** and its logical flow to anyone in the room.

Example: you can probably guess what is being modelled here...

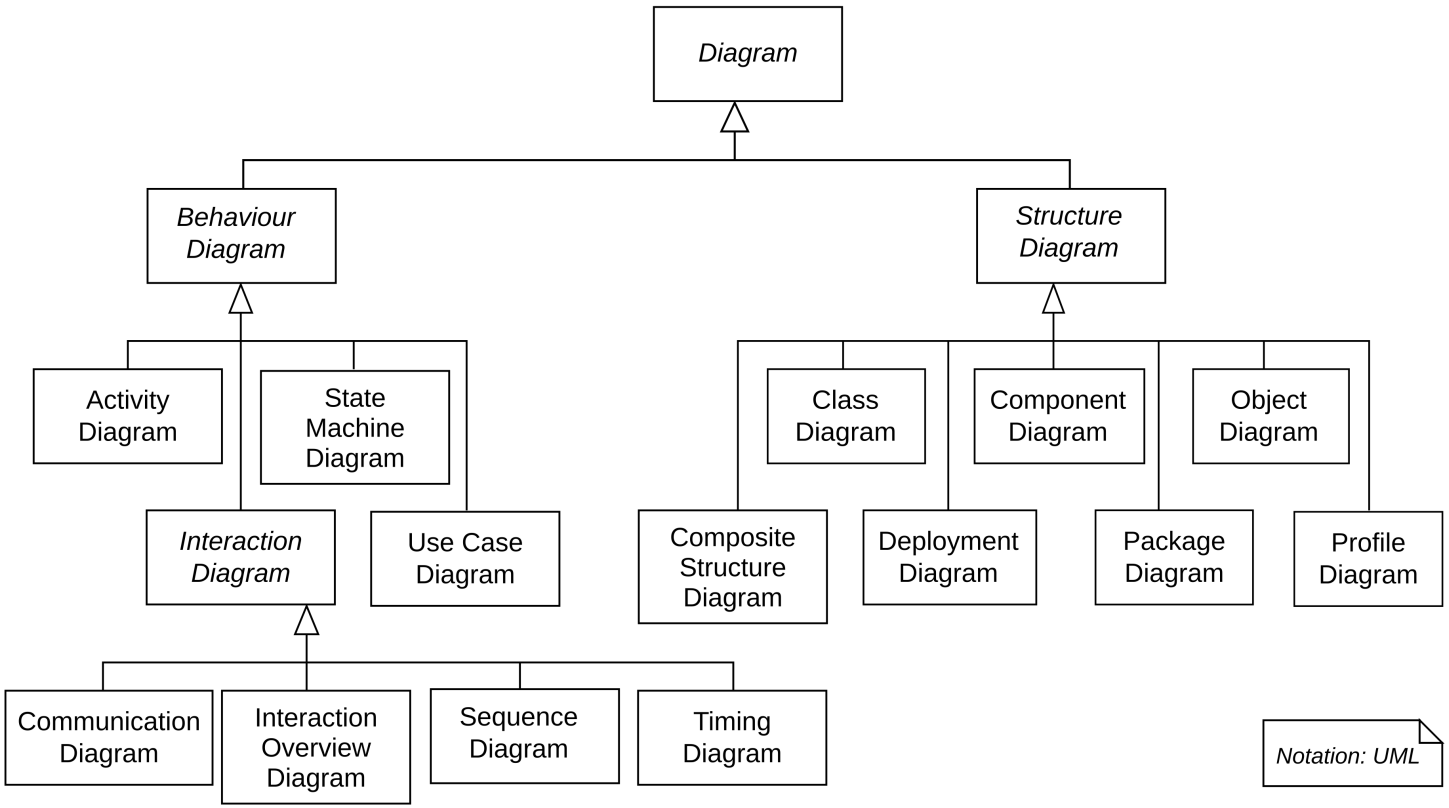


- You instantly understand the **architecture** (Phone, Auth, Media).
- You understand the **logic** (You cannot stream audio without verifying the subscription first).

Agenda

The Unified Modeling Language (UML)

UML is the industry standard for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system.



Key UML Diagrams for Software Modelling

UML has 14 official diagram types, but not all are equally useful for software system modelling. Some are more relevant for hardware design, business process modelling, or database design.

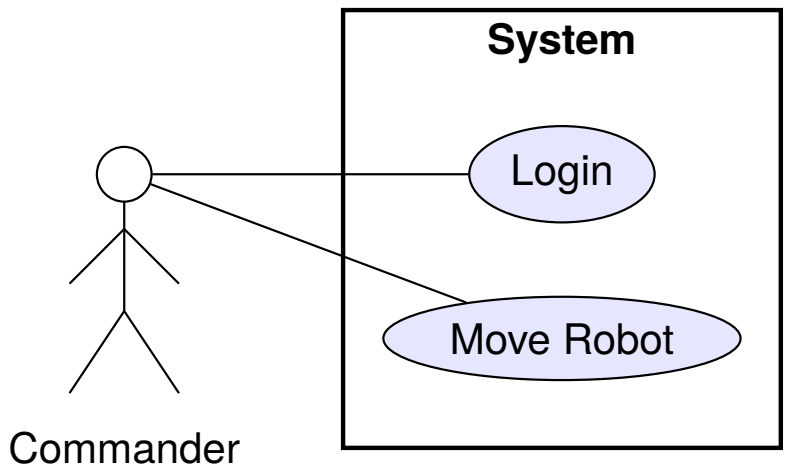
Top 4 Most Used UML Diagrams:

1. **Use Case Diagrams** (Interaction/External)
2. **Sequence Diagrams** (Interaction)
3. **Class Diagrams** (Structural / OOP)
4. **Activity Diagrams** (Behavioral)

1. Use Case Diagrams

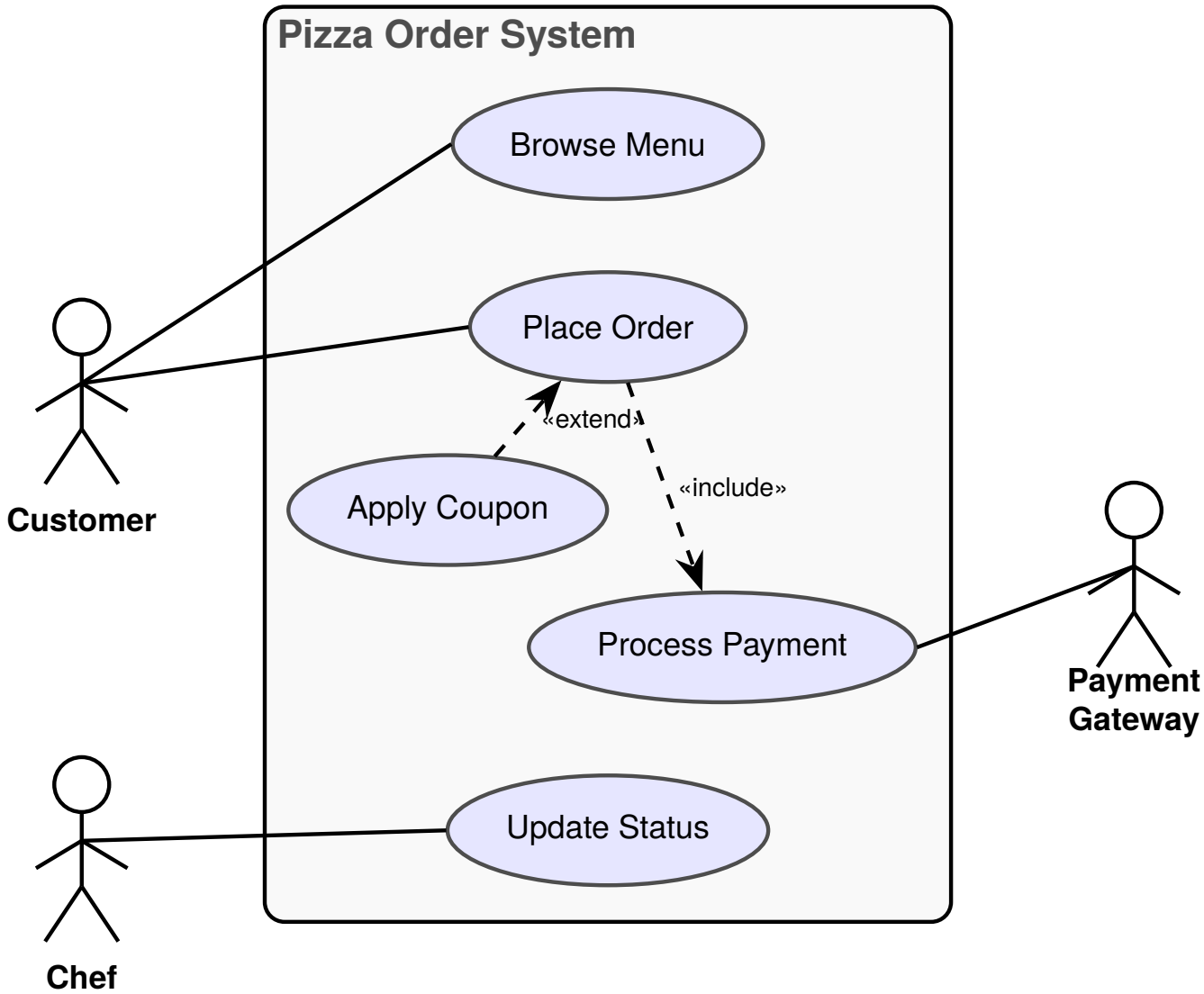
Show the interactions between a system and its environment (users or other systems).

- **Actors:** Roles played by human users or external hardware/systems (stick figures).
- **Use Cases:** The goal or action (ovals).
- **System Boundary:** The scope of your software (box).



1. Use Case Diagrams

Example: an online pizza Delivery System

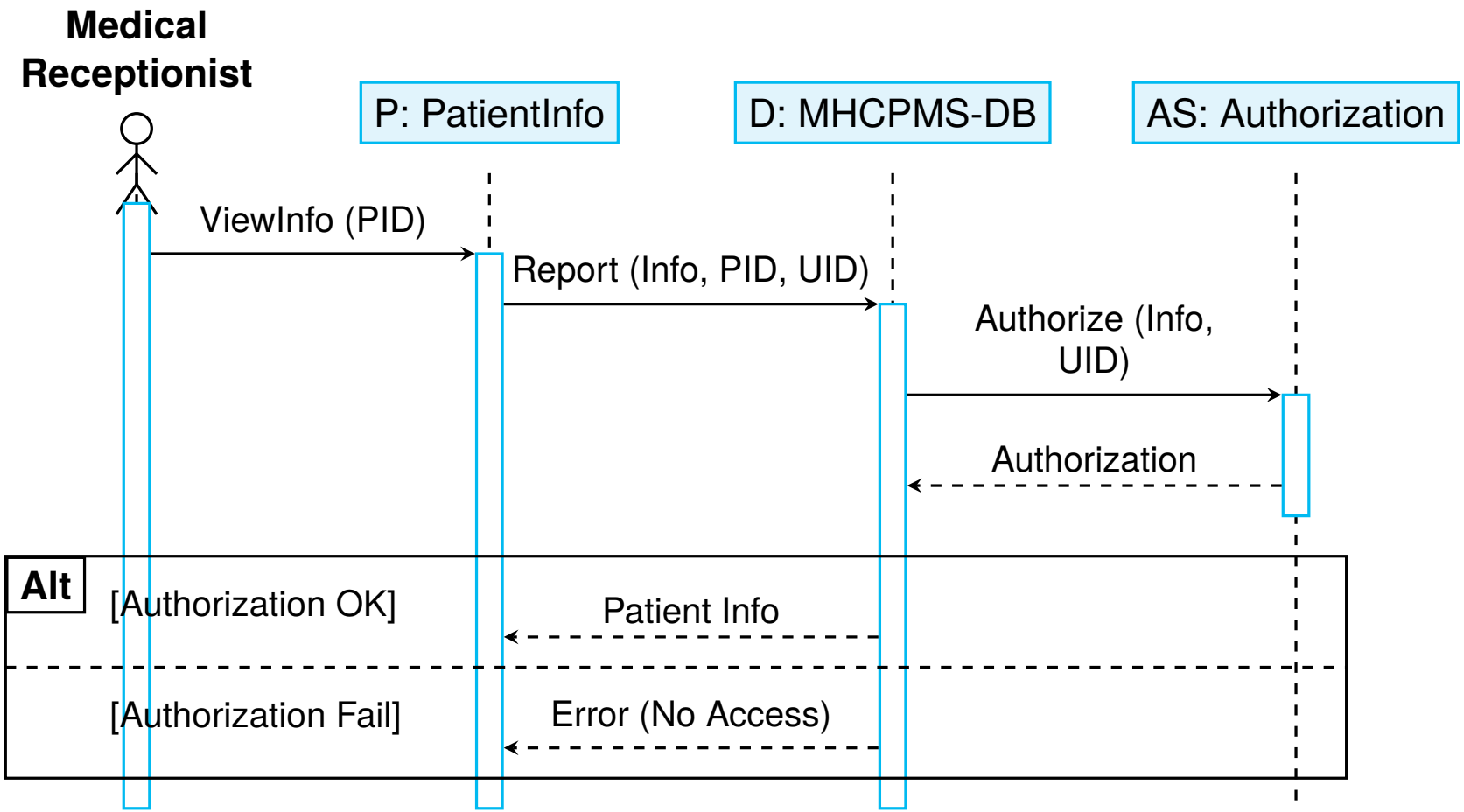


2. Sequence Diagrams

- Sequence diagrams in the UML are primarily used to model the interactions between the actors and the objects in a system and the interactions between them.
- As the name implies, a sequence diagram shows the **sequence of interactions during a particular use case** or use case instance.
- The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these.

2. Sequence Diagrams: Medical Receptionist

Sequence diagrams model the sequence of interactions during a particular use case.



2. Sequence Diagram Components

How do we read the Medical Receptionist model? You read the sequence of interactions from top to bottom.

- **Actors & Objects:** Listed along the top. A dotted vertical line drops down from each, representing the object's *lifeline*.
- **Activation Boxes (Rectangles):** Placed on the lifeline to indicate the time the object instance is actively involved in the computation.
- **Messages (Annotated Arrows):** Indicate calls to objects. The annotations show parameters (e.g., `PID`) and return values.
- **Alternatives (Alt Box):** Used to denote conditional logic. The box is divided into sections with conditions in square brackets (e.g., `[Authorization OK]`).

Note: You don't have to include every single interaction; focus on the critical logic!

3. Class Diagrams & Object-Oriented Programming

Class diagrams are the backbone of Object-Oriented Programming (OOP). They show the object classes in the system and their associations.

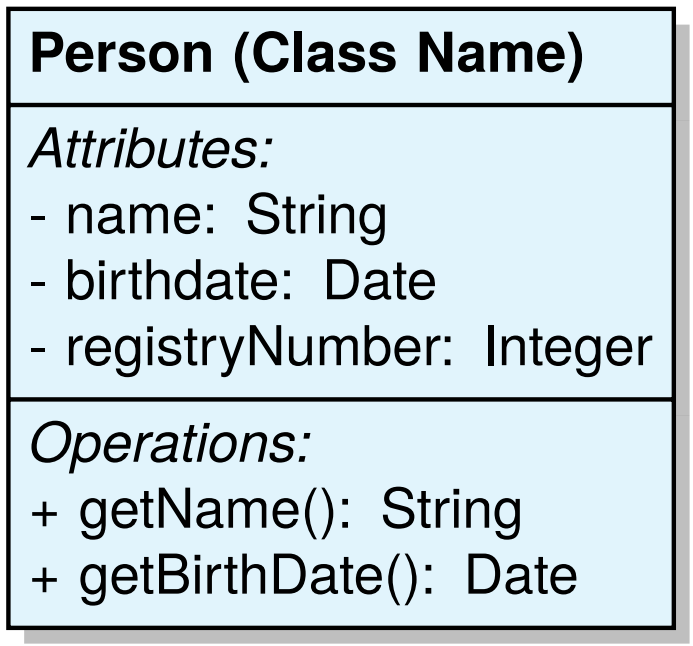
Mapping OOP to UML:

- **Encapsulation:** A class groups Data (Attributes) and Behavior (Methods/Operations) together.
- **Inheritance:** ”Is-a” relationship (e.g., a `Drone` is a type of `Robot`). Shown with a hollow arrow.
- **Association:** ”Has-a” or ”Uses-a” relationship (e.g., a `Commander` controls a `Robot`).

3. Class Diagrams: Attributes & Operations

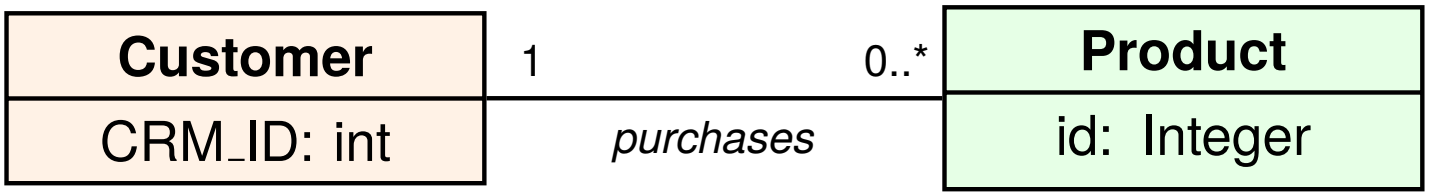
Class diagrams describe the types of objects within the system and their relationships.

- Rectangles are divided into three parts: class name, attributes, and operations (methods).
- **Attributes** characterize the class and define the state of objects within the class.
- **Visibility Modifiers:**
  - + Public (Accessible anywhere)
  - − Private (Encapsulated/Hidden)
  - # Protected (Accessible to subclasses)



3. Class Diagrams: Associations & Multiplicity

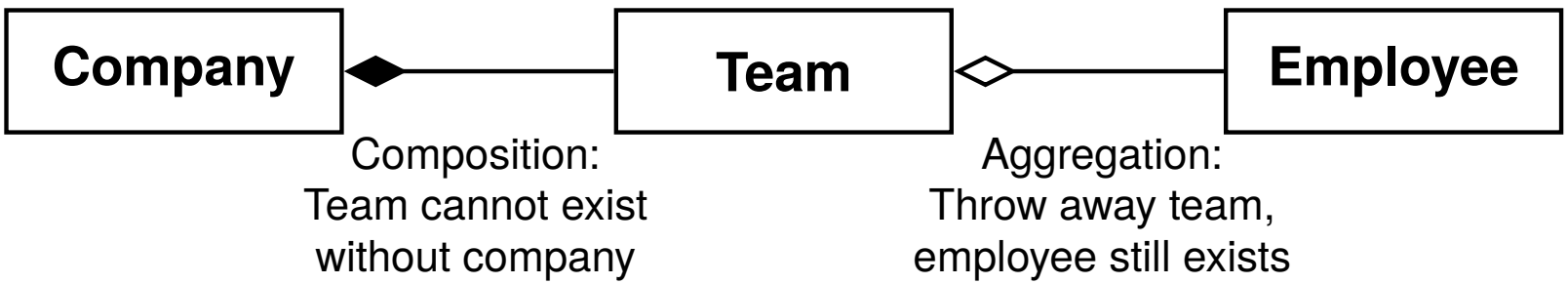
- **Associations:** Relationships between two classes are shown as lines connecting them, often with a label to describe the relationship.
- **Multiplicity:** The cardinality of the relationship (how many instances of a class can be associated with another) is represented at the ends of an association line.



1 = Exactly one
0..* = Zero or many
1..* = One or many

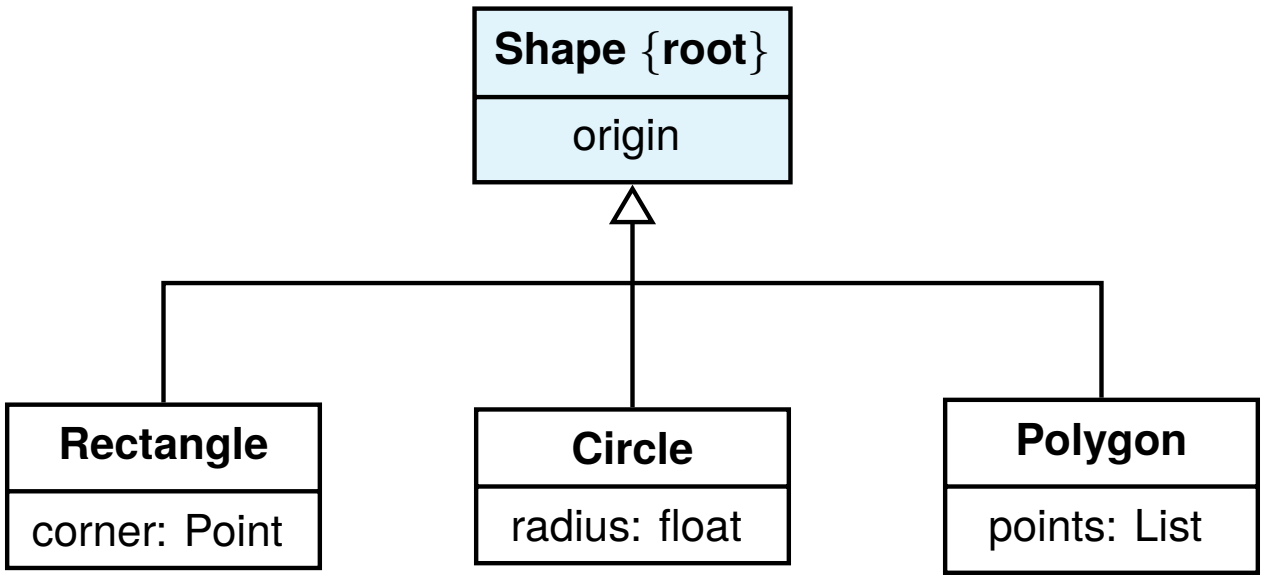
3. Class Diagrams: Aggregation vs Composition

- **Aggregation:** A ”whole-part” relationship where the component part can exist independently. Represented as a hollow diamond at the aggregate (whole) end.
- **Composition:** A stronger form indicating the part cannot exist without the whole. Depicted as a filled diamond at the whole end.



3. Class Diagrams: Inheritance (Generalization)

- **Inheritance:** Represents a relationship where one class (the subclass) is based on another class (the superclass), inheriting its features.
- Depicted as a line with a **hollow triangle** pointing towards the superclass.



4. Activity Diagrams

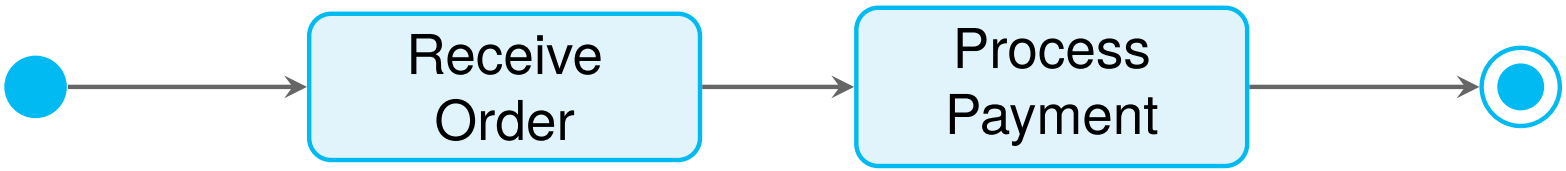
Activity diagrams show the sequence of activities involved in a process or data processing. They are similar to flowcharts.

- Good for mapping out complex business logic.
- Uses **Decision Nodes** (diamonds) for branching logic (e.g., “Is Battery $>$ 10%?”).
- Uses **Forks** and **Joins** (thick black bars) to show concurrent, parallel processing.

4. Activity Diagrams: Basics

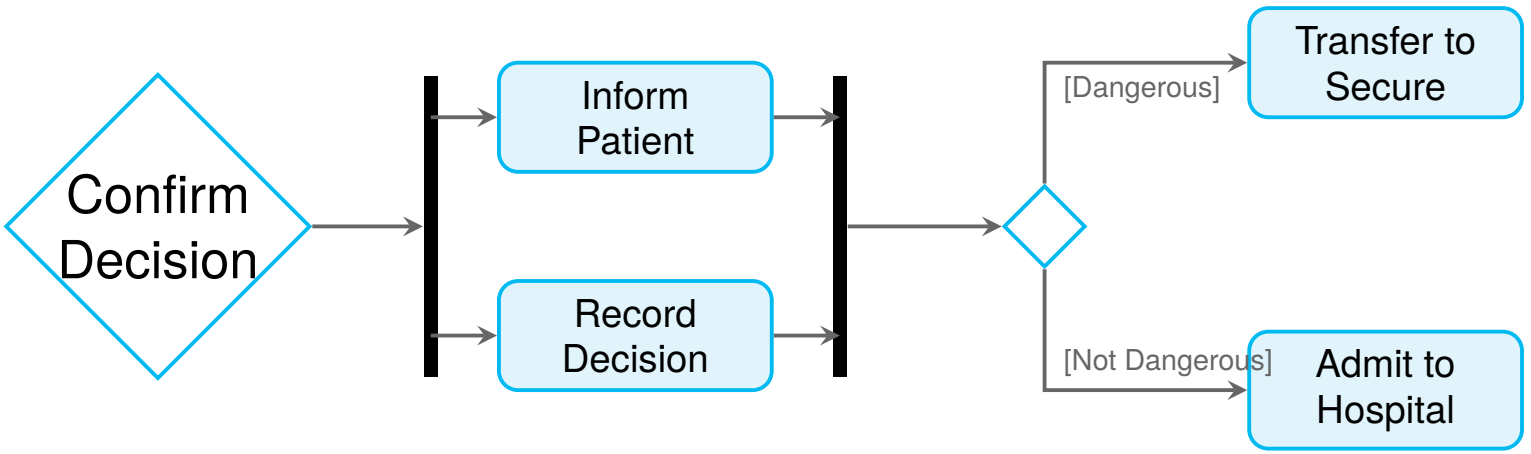
Activity diagrams show the sequence of activities involved in a process or data processing.

- **Start/End:** The start of a process is indicated by a filled circle; the end by a filled circle inside another circle.
- **Activities:** Rectangles with round corners represent the specific sub-processes that must be carried out.
- **Flow:** Arrows represent the flow of work from one activity to another.



4. Activity Diagrams: Coordination & Decisions

- **Coordination (Forks/Joins):** A solid bar indicates activity coordination. When flows lead to a solid bar, all of these activities must be complete before progress is possible.
- **Decisions (Guards):** Arrows may be annotated with guards (in square brackets) that indicate the condition when that flow is taken (e.g., `[Dangerous]`).



Summary of UML Diagrams

We have explored four key UML diagrams, each providing a unique perspective on the software system:

- **Use Case Diagrams (External Perspective):** Model the context of the system and its interactions with outside actors.
- **Sequence Diagrams (Interaction Perspective):** Detail how objects interact over time to complete a specific use case.
- **Class Diagrams (Structural Perspective):** Define the static structure, showing classes, attributes, operations, and relationships.
- **Activity Diagrams (Behavioral Perspective):** Illustrate the dynamic flow of control or data from activity to activity.

No single diagram captures everything. A complete system model requires multiple views!

When to Use Which Diagram?

Diagram	Focus	When to Use?	Key Elements
Use Case	External / Context	To capture system requirements and show what the system does for its users.	Actors, Use Cases, Boundaries
Sequence	Interaction / Time	To design the step-by-step message flow between objects for a specific scenario.	Lifelines, Messages, Activation Boxes
Class	Structural / OOP	To design the static architecture, database schema, or object-oriented code structure.	Classes, Attributes, Methods, Associations
Activity	Behavioral / Flow	To model complex business logic, parallel processes, or algorithms.	Activities, Decisions, Forks, Joins

Agenda

Interactive Exercise: The Scenario

Scenario: ”PizzaOps” Online Delivery System

We are building a software system for a local pizza shop.

- **Customers** can log in, browse the menu, and place an order.
- **Chefs** receive the order in the kitchen and update its status to ”Baking”.
- Payments are processed securely via a **Third-Party Payment Gateway** (like Stripe).

Question to the room: If we want to map out WHO uses this system and WHAT they can do, which diagram do we start with?

Step 1: The Use Case Diagram

Task: Identify the Actors and Use Cases.

Question to the room: Who are the actors and what are the use cases?

Step 1: The Use Case Diagram (Solution)

Actors:

- Customer (Human)
- Chef (Human)
- Payment Gateway (External System)

Use Cases (What are the bubbles?):

- Browse Menu
- Place Order
- Process Payment
- Update Order Status

Step 2: The Sequence Diagram

Task: Let's trace the dynamic timeline of "Placing an Order".

Question to the room: What are the "Lifelines" (vertical lines) going to be across the top?

Step 2: The Sequence Diagram (Solution)

Timeline Draft:

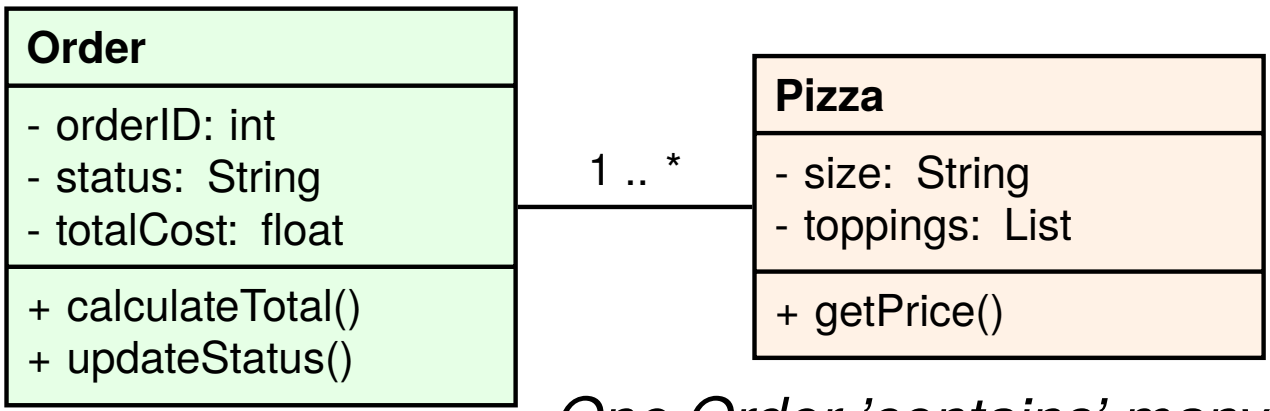
1. `Customer` sends ‘submitOrder()‘ to `WebInterface`.
2. `WebInterface` sends ‘processCharge()‘ to `PaymentGateway`.
3. `PaymentGateway` returns ‘Payment Success‘.
4. `WebInterface` sends ‘createTicket()‘ to `KitchenSystem`.

Step 3: The Class Diagram (OOP)

Task: Identify the structural Classes, Attributes, and Methods.

Question to the room: What are the main classes and their relationships?

Step 3: The Class Diagram (OOP) (Solution)



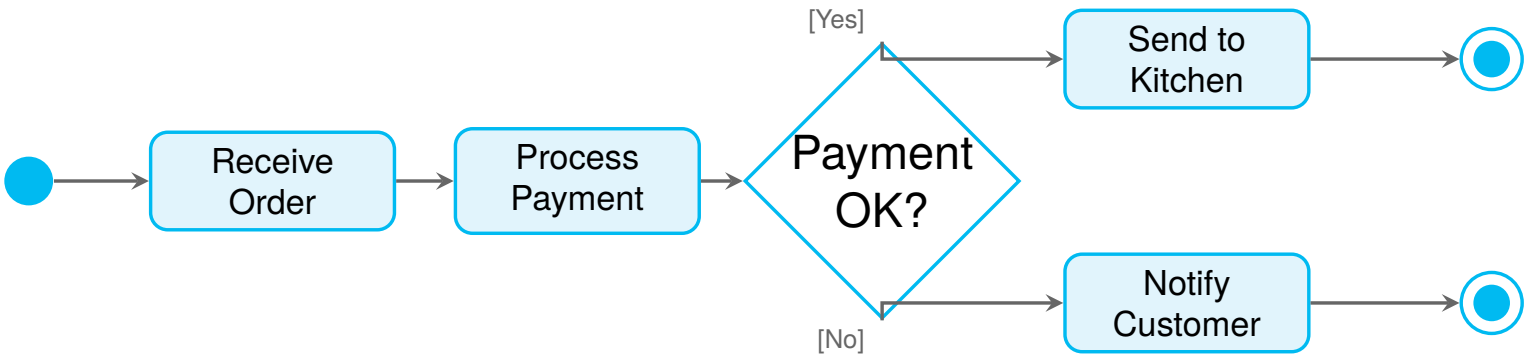
One Order 'contains' many Pizzas

Step 4: The Activity Diagram

Task: Map out the business logic for ”Processing an Order”.

Question to the room: What happens if the payment fails?

Step 4: The Activity Diagram (Solution)



Agenda

Reading & Resources

- **Recommended Reading:** Sommerville, *Software Engineering 9th Edition*, Chapter 5 (System Modeling).
- **Next Week:** We will cover **Software Patterns and Reuse**.

Any Questions?

fdelduchetto@lincoln.ac.uk